



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

واحد محاسبه با امکان انتخاب ثبات مبدا و مقصد

نام درس

آزمایشگاه معماری کامپیوتر

نیم سال ۱۴۰۴۳

استاد

دکتر سربازی آزاد

گروه ۲

امیرشایان سلیمانی نیا - ۴۰۳۱۰۶۱۰۸

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	مقدمه و هدف آزمایش	۲
۲	شرح آزمایش	۲
۳	ساختار و مراحل طراحی	۳
۱.۳	زیرمدار ALU	۳
۲.۳	زیرمدار Reg_File	۳
۳.۳	مدار Main	۴
۴	تست مدار	۵
۱.۴	مرحله اول	۵
۲.۴	مرحله دوم	۶
۳.۴	مرحله سوم	۶
۴.۴	مرحله چهارم	۷
۵	نتیجه گیری	۷

۱ مقدمه و هدف آزمایش

هدف این آزمایش، طراحی بخش‌های اولیه یک کامپیوتر ساده شامل ALU و مجموعه رجیسترها است تا در نهایت بتوانیم برنامه‌های زبان ماشین را روی آن اجرا کنیم.

۲ شرح آزمایش

در این آزمایش یک پردازنده ۸ بیتی پایه را در Logisim طراحی میکنیم که شامل ۴ رجیستر عمومی (R0 تا R3) و یک ALU با قابلیت انجام عملیات جمع و تفریق است. در این Datapath، یکی از عملوندها (عملوند اول) همواره رجیستر R0 است و کنترل کل مسیر توسط یک دستور ۶ بیتی انجام می‌شود.

مدار طراحی شده شامل ورودی‌ها و خروجی‌های زیر است:

ورودی‌ها:

- دستور ۶ بیتی (Inst): این ورودی کنترل کل مدار را بر عهده دارد و از طریق یک Splitter به سه بخش تقسیم می‌شود:

- بیت ۵ (Op): تعیین‌کننده نوع عملیات (۰ برای جمع، ۱ برای تفریق).

- بیت‌های ۴ و ۳ (Dest): انتخاب رجیستر مقصد (یکی از رجیسترهای R0 تا R3) برای ذخیره نتیجه نهایی.

- بیت‌های ۲ تا ۰ (Source): انتخاب عملوند دوم با استفاده از یک Multiplexer، که می‌تواند محتوای یکی از رجیسترها یا مقادیر ثابت ۰، ۱ و ۱- باشد.

- سیگنال Clock: به دلیل ترتیبی بودن مدار از آن برای پایه کلاک در رجیسترها استفاده میشود.

- سیگنال Clr: این سیگنال در دستور کار وجود نداشت و فقط برای ساده‌تر شدن تست و ریست شدن مدار، به مدار اضافه شده است.

خروجی‌ها:

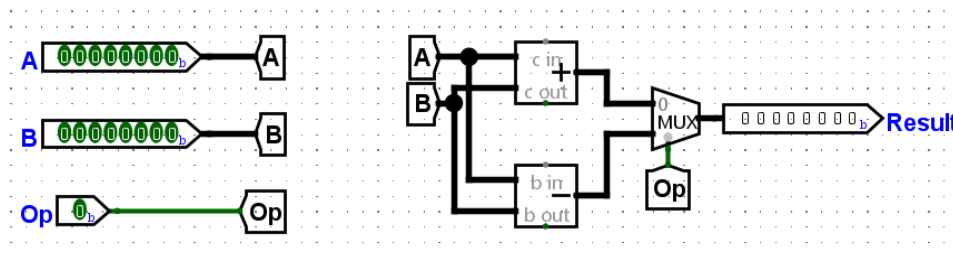
- محتوای رجیسترها (R0 تا R3): خروجی اصلی و نهایی این مدار، پین‌های ۸ بیتی متصل به رجیسترها هستند.

۳ ساختار و مراحل طراحی

مدار از دو زیر مدار و یک بخش اصلی تشکیل شده است در ادامه مراحل پیاده سازی هر بخش توضیح داده شده است:

۱.۳ زیرمدار ALU

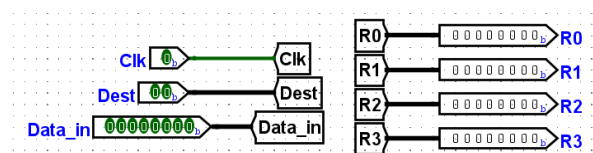
این بخش برای انجام محاسبات می باشد. از دو قطعه Adder و Subtractor هشت بیتی استفاده کردیم که ورودی های A و B به هر دوی آنها متصل شده اند. برای انتخاب بین نتیجه جمع یا تفریق، خروجی این دو قطعه را به یک Multiplexer وصل کردیم که توسط پین کنترلی Op مدیریت می شود؛ در صورت یک بودن سیگنال Op، باید عملیات تفریق انجام شود و در غیر این صورت، حاصل عملیات جمع به خروجی می رود.



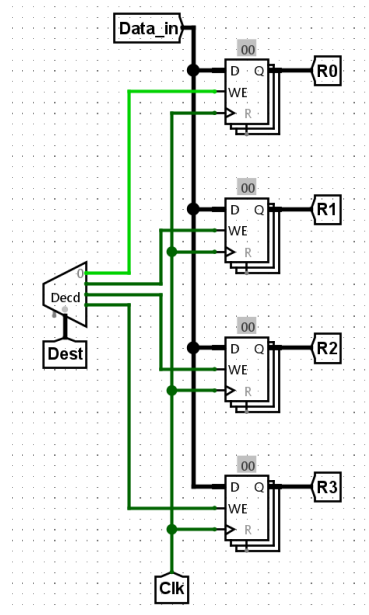
شکل ۱: زیرمدار ALU برای عملیات جمع و تفریق

۲.۳ زیرمدار Reg_File

این زیرمدار وظیفه نگهداری داده ها را بر عهده دارد. از چهار قطعه Register هشت بیتی استفاده کردیم. Data_in به پایه ی داده ی تمام رجیسترها متصل است. برای اینکه در هر سیکل فقط رجیستر هدف به روزرسانی شود، از یک Decoder استفاده کردیم که ورودی Dest را دریافت کرده و فقط پایه Enable رجیستر مورد نظر را فعال می کند. سیگنال Clock نیز به همه رجیسترها متصل شده است.



شکل ۲: ورودی ها و خروجی ها



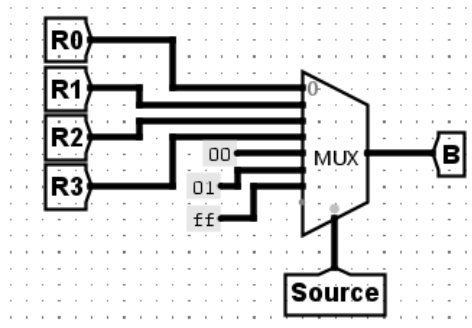
شکل ۳: انتخاب رجیسترها با دیکدر

۳.۳ مدار Main

مدار اصلی از به هم پیوستن زیرمدارها و ایجاد Datapath تشکیل شده است. با توجه به معماری خواسته شده، این مدار را در دو بخش مجزا بررسی می‌کنیم:

بخش اول: انتخاب عملوند دوم

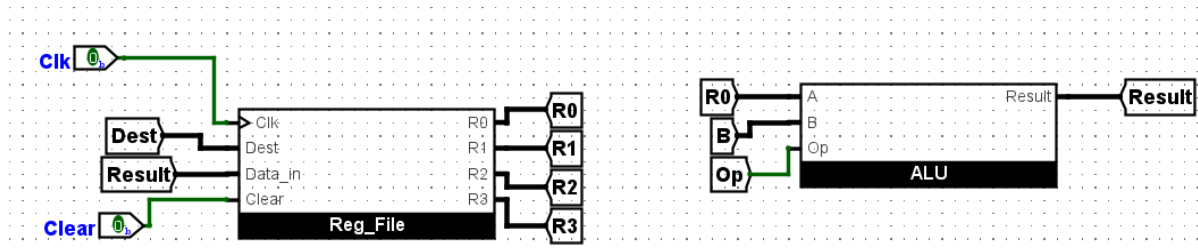
عملوند دوم (B) می‌تواند حالات مختلفی داشته باشد. برای این مسیر از یک Multiplexer استفاده کردیم. سیگنال Source تعیین می‌کند که مقدار ورودی دوم به ALU، محتوای یکی از رجیسترها باشد یا یکی از مقادیر ثابت ۰ (0x00)، ۱ (0x01) و -۱ (0xFF). همچنین بر اساس معماری خواسته شده، عملوند اول (A) نیز همواره به صورت ثابت به خروجی رجیستر R0 متصل است.



شکل ۴: مسیر انتخاب B

بخش دوم: پردازش نتیجه

عمل اصلی پردازش در این بخش و توسط دو زیرمدار ALU و Reg_File انجام می‌شود. عملوندهای و B به همراه سیگنال Op وارد واحد ALU می‌شوند. خروجی نهایی (Result) در یک حلقه به Reg_File بازگردانده می‌شود تا در رجیستر Dest ذخیره گردد.



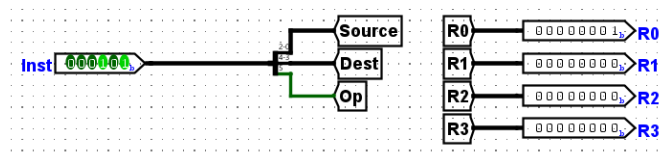
شکل ۵: اتصال زیرمدارها و تکمیل حلقه Datapath

۴ تست مدار

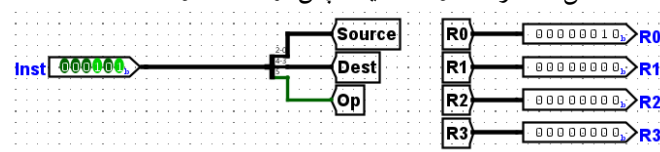
با توجه به اینکه عملوند اول مدار همواره رجیستر R0 است، برای تست صحت عملکرد مدار ۴ مرحله متوالی زیر را در نظر می‌گیریم (فرض بر این است که در ابتدا تمام رجیسترها صفر هستند):

۱.۴ مرحله اول

در این مرحله هدف ما جمع کردن مقدار ثابت ۱ با محتوای R0 و ذخیره آن در خود R0 است. دستور ۶ بیتی 000101 را در ورودی Inst وارد می‌کنیم (Op=0 برای جمع، Dest=00 برای R0 و Source=101 برای ثابت ۱). برای این مرحله دو پالس Clock اعمال می‌کنیم. پس از کلاک اول مقدار R0 برابر با ۱ و پس از کلاک دوم برابر با ۲ (00000010) می‌شود.



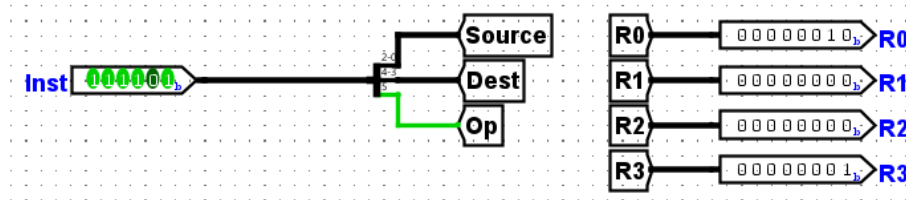
شکل ۶: مرحله اول - نتیجه پس از کلاک اول (R0=1)



شکل ۷: مرحله اول - نتیجه پس از کلاک دوم (R0=2)

۲.۴ مرحله دوم

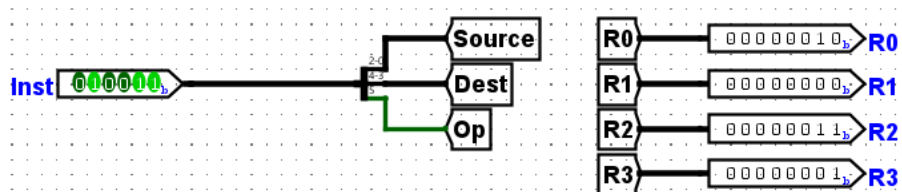
در این مرحله می‌خواهیم عدد ثابت ۱ را از مقدار فعلی R0 (که اکنون ۲ است) کم کرده و نتیجه را در رجیستر R3 ذخیره کنیم. دستور 111101 را به ورودی می‌دهیم (Op=1 برای تفریق، Dest=11 برای R3). با اعمال یک پالس Clock، حاصل عبارت (۲-۱) در رجیستر R3 ذخیره شده و مقدار آن ۱ (00000001) می‌شود.



شکل ۸: مرحله دوم - تفریق ثابت ۱ از R0 و ذخیره در R3

۳.۴ مرحله سوم

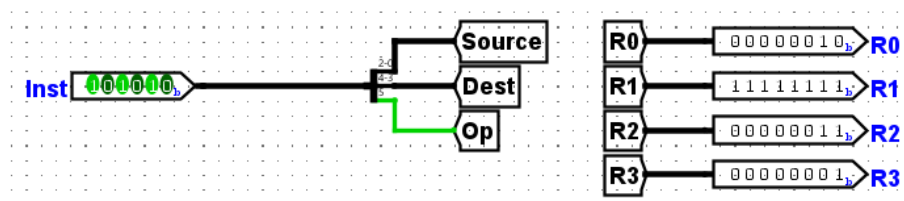
اکنون قصد داریم مقدار فعلی R3 (که برابر با ۱ است) را با R0 (که همچنان ۲ است) جمع کرده و در رجیستر R2 ذخیره کنیم. برای این کار دستور 010011 را وارد می‌کنیم (Source=011 و Dest=10). با زدن یک پالس Clock، حاصل جمع (۲ + ۱) محاسبه شده و خروجی R2 برابر با ۳ (00000011) می‌شود.



شکل ۹: مرحله سوم - جمع محتوای R3 با R0 و ذخیره در R2

۴.۴ مرحله چهارم

در مرحله نهایی، می‌خواهیم محتوای R2 (که در مرحله قبل ۳ شد) را از R0 (که مقدارش ۲ است) تفریق کنیم و نتیجه را در R1 قرار دهیم. دستور 101010 را اعمال کرده و یک پالس Clock می‌زنیم. از نظر ریاضی حاصل تفریق (۲-۳) برابر با منفی یک خواهد بود که مدار این نتیجه را به درستی محاسبه کرده و مقدار 11111111 را در رجیستر R1 ذخیره می‌کند.



شکل ۱۰: مرحله چهارم - تفریق محتوای R2 از R0 و تولید عدد منفی در R1

۵ نتیجه‌گیری

در این آزمایش، ما توانستیم معماری پایه یک کامپیوتر ساده شامل واحد محاسبات و مجموعه رجیسترهای ۸ بیتی را طراحی و پیاده‌سازی کنیم. با تقسیم کردن مدار به بلوک‌های مختلف مثل ALU و Reg_File، توانستیم کارها را به بخش‌های کوچک‌تر و قابل مدیریت تقسیم کنیم. مدار ما بر اساس دستورات ۶ بیتی (Inst) عملیات مورد نظر را انتخاب می‌کند و با اعمال پالس سیگنال Clock، نتایج را به درستی در رجیستر مقصد ذخیره و به‌روزرسانی می‌کند.